2026 January Term - 50.051 : Programming Language Concepts Project Report

Group 7

1006627 Joshua Chua Tze Ern

1006852 Goh Yi Shen

1006892 Chew Ming Hui

1004435 Jun Jin

1006952 Anika Ajay Handigol

1. Project Overview

Our project is focused on building a compiler in ANSI C that converts Markdown files into Latex files. The scope of the project focused on being able to convert common Markdown structures, such as headings, paragraphs, formatting, and document sections, into equivalent LaTeX commands.

The program works directly with files throughout the process. A user provides a Markdown file, and the system reads the text line by line or character by character. After interpreting the Markdown structure, it uses parser logic, and state-based processing to manage different parts of the conversion and writes the converted LaTeX content into a separate output file. For features such as embedded images, the compiler is able to handle both text-based document conversion and binary file copying into the output directory.

2. Project Requirements and Constraints

Our project satisfies the project constraint of processing files as an input and output at the end and during the programme runtime by implementing the full Markdown-to-LaTeX conversion pipeline in C. To implement our compiler, we first defined a tokeniser that extracts the full list of tokens available in the Markdown syntax. These tokens include headings, emphasis markers, links, images, math delimiters, lists, and blockquotes. We then defined a parser that uses these tokens to understand the input document structure and generates the matching LaTeX code. Invalid Markdown input can be detected during parsing, allowing the program to report errors instead of silently producing incorrect output.

The state machine logic in our compiler is implemented mainly through the lexer’s token scanning process. As the lexer reads Markdown input, it keeps track of its current position using variables such as the character index, line number, column number, text buffer, and buffer length. When the lexer is reading normal text, it stays in a “text accumulation” state and stores

characters in the buffer. When it encounters a special Markdown character which include non-alphanumeric characters or a newline, it flushes any accumulated text as a **TOK_TEXT** token and switches into token recognition logic for that symbol. Some symbols also require looking ahead to the next characters, such as detecting *, **, and ***, or $ and $$. After emitting the correct token, the lexer updates its line and column state and continues scanning. This behaves like a simple state machine due to the lexer's actions depending on its current scanning state and the token it is processing. We implemented our own lexer and parser instead of relying on external parsing libraries.

Overall, the project uses clean C programming to fulfill the project constraints of file reading and writing, state-machine-based lexing, and the building of an original, custom parser. The implementation does not depend on external libraries and keeps the compiler logic within the project's own C source files.

3. Challenges

One challenge we had to address was making sure that the project could run correctly on each team member's computer. Since the group was working across different environments, we needed to account for operating system differences, especially between Windows and Unix-like systems. This led to fixes such as the Windows compatibility update in the codebase, including handling directory creation in a platform-dependent way. We also used the Makefile to standardise the compilation process so that everyone could build the project in a consistent manner. In addition, we tested the program on different members' machines using the same Markdown inputs and checked that the generated token output and LaTeX output were consistent across setups.

During development, one major difficulty was that Markdown syntax did not fit neatly into an LL(k) parsing table approach. Markdown is flexible and context-dependent, so the same symbol can have because it could not simply replace symbols one by one. It had to understand the structure's different meanings depending on where it appears in the line or document. For example, * can represent italic text, bold text, or part of normal text, while - can represent a list item or just a dash. Because of this, building a fixed LL(k) table would have been too rigid and difficult to maintain. Instead, the project had to rely on custom lexer and parser logic that checks the surrounding context while scanning the input.

Another challenge was the large number of Markdown token types. Even a limited Markdown subset includes headings, bold text, italic text, inline code, code blocks, links, images, math mode, blockquotes, lists, and normal text. Each of these features has different opening and closing markers, and some of them can be nested inside others. This made the parser more complex than expected, of the document and to decide how each token should be converted into valid LaTeX.

The table feature was especially difficult to lex and parse. Markdown tables use repeated symbols such as pipes |, dashes -, and newlines, but these characters can also appear in normal text. The parser needs to identify whether a group of lines is really a table, separate the

header row, alignment row, and body rows, and then convert them into the correct LaTeX table structure. This requires looking across multiple lines instead of processing each token independently, which makes table handling one of the more complicated parts of the development process.

We also mitigated with the bug that once generate_latex() is called with the root NODE_DOCUMENT, the entire output is produced through a single recursive traversal with no revisiting and no backtracking. emit_block() and emit_inline() walk the tree top-down, and because every node was fully resolved during parsing, neither function needs to make decisions more complex than switching on node type. The traversal follows the same parent-to-child direction that the parser used to build the tree: NODE_DOCUMENT at the root, block nodes as its direct children, inline nodes as children of those blocks, and NODE_TEXT leaves at the bottom of each branch. emit_block() handles the structural layer and hands off to emit_inline() where block structure ends and inline content begins. emit_inline() then recurses into its own children for nested formatting such as bold inside italic, bottoming out when it reaches a NODE_TEXT leaf and calls escape_latex() on its value. One special case is NODE_TABLE, where emit_block() directly calls emit_inline() for each table cell's children because table cells store inline content without an intermediate block node. Lists are handled through NODE_LIST and NODE_ITEM, allowing both list item text and nested block content to be emitted correctly.

4. Code

4.1 main.c

main.c is the entry point and orchestrator for the entire pipeline. It is responsible for reading the input file, passing it through each stage in order, and writing the final outputs. It does not contain any parsing or emission logic itself, as it stitches the stages together and manages the file handles around them.

4.1.1 read_file()

read_file() takes a file path and returns a heap-allocated null-terminated string containing the full contents of the file. It opens the file in binary mode, seeks to the end to determine the file size with **ftell()**, rewinds, then mallocs a buffer of **size + 1** to account for the null terminator. The contents are read in a single **fread()** call, and the null terminator is written manually at the end. The returned buffer is the caller's responsibility to free, which **main()** does before exiting.

4.1.2 ensure_output_dir()

ensure_output_dir() calls ***mkdir*** to create the output directory before any output files are written. The permissions value 0755 means the owner has read, write, and execute access while group and others have read and execute only. If the directory already exists, that case is accepted and not treated as an error.

4.1.3 main()

main() enforces that exactly one argument is provided beyond the program name. argv[0] is always the program name itself and argv[1] is the input file path. If **argc** is not 2, a usage message is printed and the program returns immediately.

From there the pipeline runs in a fixed sequence:

1. **File Load**
    a. **read_file()** loads the source into a heap buffer.
2. **Tokenization**
    a. **lex()** tokenizes that buffer into a statically allocated **TokenList**.
    b. The token count is printed to **stdout**
    c. Token stream is written to output/tokens.txt using print_tokens() for debugging.
    d. The file (output/tokens.txt) is then closed.
3. **Parsing**
    a. The parser is initialized by setting parser.tokens to point to the token list and parser.pos to 0.
    b. **parse_document()** is called and returns the completed root ASTNode.(At this point the entire AST is resident in memory)
    c. A second output file, output/output.tex, is opened
4. **Output Generation**
    a. **generate_latex()** walks the AST and writes the LaTeX output to that file
5. **Post-process**
    a. The file (output/output.tex) is closed, completion messages are printed
    b. **file_src** is freed before **main()** returns
    c. The AST itself is not freed before exit, as discussed in the conclusion

4.2 Lexer.c

The lexer is the first stage of the pipeline. It takes the raw markdown source string and converts it into a flat array of typed tokens stored in a TokenList.

4.2.1 is_special()

is_special() is a simple character classifier that returns 1 if a character has potential syntactic meaning in Markdown and 0 otherwise. The characters it recognises are #, *, _, `, [,], (,), !, $, newline, space, tab, >, -, |, and backslash. Every character not in this set is treated as plain text and accumulated into the text buffer. This function is the gate between text accumulation and token emission.

4.2.2 push()

push() writes a single token into the next available slot in the **TokenList**. It checks the count against **MAX_TOKENS** first and silently returns if the list is full, which is the only capacity guard in the lexer. It then writes the token type, line number, column number, and value into the slot using **strncpy** with a hard limit of **MAX_TOKEN_VALUE** minus one, then manually sets the final byte to the null terminator. The line and column values are tracked by **lex()** and passed into **push()**. This allows each token to carry source-location metadata for debugging output in **print_tokens()**. In the current implementation, this metadata is not used by the parser.

4.2.3 flush_text()

flush_text() is called whenever the lexer is about to emit a special token, and again at the end of the source. If the text buffer has accumulated any characters, it null-terminates the buffer and calls **push()** to emit a **TOK_TEXT** token containing everything accumulated so far, then resets the length counter to zero. If the buffer is empty it returns immediately without pushing anything. This function is what ensures that plain text between special characters is not lost. The lexer accumulates it character by character and flushes it as a single token the moment a special character is encountered.

4.2.4 lex()

lex() is the main lexer function. It iterates through the source string one character at a time using an index variable i, tracking the current line and column as it goes. For each character it reads the current character c, one character ahead as next, and two characters ahead as after. These lookahead values allow the lexer to recognise multi-character tokens like **, ***, ```, and $$ without backtracking.

If the current character is not special, it is appended to the text buffer and the loop continues. If it is special, **flush_text()** is called first to emit any accumulated text, then the character is matched in a brute-force if-else chain.

Whitespaces (spaces and tabs) are treated as a special case. Even though **is_special()** flags them, the lexer does not emit them as distinct token types. Instead, it appends them directly to the text buffer just as plain characters would be, which means they flow through as part of **TOK_TEXT** tokens. This is a deliberate choice that simplifies the parser, which uses

whitespace-only text tokens to measure list indentation rather than needing a dedicated whitespace token type.

Newlines emit a **TOK_NEWLINE** token, increment the line counter, and reset the column to 1. The star, underscore, backtick, and dollar characters each check their lookahead values to decide between single, double, or triple token variants. The hash character is handled by running a small loop that consumes up to seven consecutive hash characters, stores them in a local buffer, and emits a single **TOK_HASH** token whose value string contains all of them. The parser then uses **strlen()** on that value to determine the heading level. Brackets, parentheses, bang, gt, dash, and pipe each emit their own single-character token type with no lookahead needed.

The backslash case is the lexer's only escape-related handling. When a backslash is encountered, the lexer consumes the following character and emits it as plain text instead of tokenizing it by its usual special meaning. This is how Markdown escape sequences like * or # are handled. The backslash is consumed and the next character is pushed as plain text.

After the loop finishes, **flush_text()** is called one final time to emit any trailing text that had not yet been flushed, and then a **TOK_EOF** token is pushed. This **EOF** token is what allows the parser to safely loop using **peek()** without ever reading past the end of the token array.

4.3 AST

To represent the parsed document structure and track the type of content being processed, we defined custom data structures in **parser.h**. Supporting helper functions for working with these structures are declared in **parser.h** and implemented in **parser.c.**

ASTNode is the core node structure used to build the abstract syntax tree. It stores a **NodeType** value, defined as an enum, to distinguish between the different kinds of parsed content such as headings, paragraphs, text, lists, links, and code blocks. Each **ASTNode** also stores a level field, an ordered flag, and a value string.

To represent hierarchical document structure, each **ASTNode** contains a fixed-size array of child-node pointers with capacity for up to 512 children, along with a **child_count** field that records how many children are currently attached.

4.4 parser.c

Once the lexer returns a list of tokens, it places that token list into a Parser struct, and passes that Parser struct into the **parse_document()** function, which is the entry point of our code for parsing the tokens into an **AST**.

The parsing is done using a recursive-descent approach with lookahead. Block-level parsing happens first, and inline syntax such as bold, italic, and links is parsed within each block as the

AST is being built. After parsing is complete, the AST is passed to the LaTeX generation stage, which walks the tree and emits the final output.

Once a block is identified, the parser continues forward through the same token stream, and the inline part of the parser is called within that block's boundary, using **parse_inline()** to identify inline aspects like bold and italics. Once the block boundary is met, the parser returns to **parse_document()** and looks for the next block. This happens repeatedly for every block, while standalone newlines are skipped, until **TOK_EOF** is reached. With this 2-step structure, block boundaries define where inline syntax is parsed, and the resulting structure is stored in the main AST, which is converted into LaTeX later.

Following the path of **parse_document()**, it initializes and declares an empty node with the type **NODE_DOCUMENT**. This is done with a separate function which mallocs and initializes default values for the **ASTNode** data structure.

This **ASTNode** is stored as a pointer, doc, which is used to hold the root of the AST as it is built up.

4.4.1 Headings

peek for headings is not a dedicated function. parse_block() directly checks if the first non-whitespace token in the block is TOK_HASH. If it is, it immediately hands off to parse_heading().

parse_heading() advances past the hash token and reads how many # characters were in it — this is stored directly in the token's value, so strlen(hash->value) gives the heading level. It then loops forward consuming every token until it hits a TOK_NEWLINE or TOK_EOF, calling parse_inline() on each one. Those inline calls build up child nodes under the heading node. A heading with bold text inside it will have a NODE_BOLD child, not raw text. When the newline is found, it is consumed with match() and the heading node is returned.

If the hash token is never found, parse_block() simply falls through to parse_paragraph().

4.4.2 Block Parsing

Once **parse_document()** has its document node set up, it enters a loop that runs until it hits TOK_EOF. Each iteration calls parse_block(), which is the main function for block-level structures. The job of parse_block() is to look at the upcoming tokens and decide what kind of block is starting.

It does this through a series of peek functions. These are non-consuming. Instead, they look ahead in the token stream without actually advancing the position. This is important because the parser sometimes needs to inspect upcoming tokens before deciding what block type it is. For example, **peek_code_block()** scans forward past any leading whitespace to check if a triple

backtick is coming up. If it is, control is handed off to **parse_code_block()**. Same goes for **peek_blockquote()**, which looks for a > token, and **peek_block_math()**, which looks for $$.

The table case is a bit more involved. **peek_table()** first checks if the current line has a pipe (|) character in it, then looks at the next line to see if it matches the separator pattern (dashes, optionally with colons for alignment). Only if both conditions hold does it commit to parsing a table. This two-line lookahead is necessary because a line with pipes is not necessarily a table.

Lists are handled through peek_list_marker(), which scans forward past indentation to find either a dash token or a number followed by a period. It also records the indentation level it found, which becomes important later when handling nested lists.

If none of the specialized peek functions match, the block falls through to **parse_paragraph()** as the default case. The heading case is slightly special. **parse_block()** checks for a **TOK_HASH** token directly, and also handles the edge case where the hash is preceded by whitespace, which it skips before dispatching to **parse_heading()**. This logic is handled in **parse_block()**, not inside **parse_paragraph()**.

4.4.3 parse_paragraph()

parse_paragraph() is the final fallback. If nothing else matched, this is what runs. It creates a NODE_PARAGRAPH and loops forward consuming tokens with parse_inline(), same as the heading, until it hits a TOK_NEWLINE or TOK_EOF. The newline is consumed and the node is returned.

The paragraph has no special opening syntax, so it cannot fail. Whatever tokens are left that don't belong to another block structure end up here.

4.5 Peek and Parse Methods

4.5.1 peek_code_block()

peek_code_block() scans forward from the current position, skipping any leading whitespace text tokens, and checks if the next non-whitespace token is TOK_TRIPLE_BACKTICK. It does not advance the parser position, only returns a boolean through an int that is 1 or 0.

If found, parse_code_block() takes over. It first skips any leading whitespace, then consumes the opening triple backtick. Everything on that same line after the backtick is consumed and discarded, since the emitter does not currently use it. The newline after that is consumed too.

From here, the parser enters a line-by-line collection loop. Each iteration saves the current position as line_start, then advances token by token until it hits a TOK_NEWLINE, TOK_EOF, or another TOK_TRIPLE_BACKTICK. Everything between line_start and that stopping point is handed to make_raw_text_from_tokens(), which concatenates all the token values in that range

into a single NODE_TEXT node. That text node is added as a child of the code block node. This repeats for every line until the closing triple backtick is found, which is then consumed along with the trailing newline.

4.5.2 peek_block_math()

peek_block_math() works identically to peek_code_block() but looks for TOK_DOUBLE_DOL instead of a triple backtick.

parse_block_math() follows the same structure as the code block parser. It skips leading whitespace, consumes the opening $$, discards the rest of that line, then collects content line by line using make_raw_text_from_tokens() until it sees another $$. Each line becomes a NODE_TEXT child of the NODE_MATH_BLOCK node. The closing $$ and trailing newline are consumed at the end.

The key difference from code blocks is in the emitter: math block children are written out as raw text between \[and \], without escaping. This is intentional as the content is LaTeX math syntax and must pass through unmodified, which makes our lives easier.

4.5.3 peek_blockquote()

peek_blockquote() scans forward past leading whitespace and checks for a TOK_GT token, only returns a boolean through an int that is 1 or 0.

parse_blockquote() runs a loop that keeps consuming lines for as long as each line starts with a >. At the start of each iteration it saves the current position. It skips whitespace, then checks for TOK_GT. If it's not there, it restores the saved position and breaks. This is important because those tokens belong to whatever comes next, not the blockquote.

If > is found, it is consumed. Any single whitespace token immediately after it is also consumed, accounting for the conventional > text formatting. Then a NODE_PARAGRAPH is created and parse_inline() is called on each token until the newline, building up inline children inside that paragraph. The paragraph is added as a child of the blockquote node, and the loop continues.

4.5.4 peek_table()

peek_table() was the hardest of the peek functions. It first calls peek_table_row() on the current line, which scans the line for any TOK_PIPE token. If there's no pipe, it returns 0 immediately. If there is a pipe, it then looks at the next line and calls is_table_separator_line() on it.

is_table_separator_line() walks through that next line and splits it into segments by pipe characters. Each segment is passed to segment_is_table_separator(), which checks that the segment contains at least one dash and optionally colons on either side for alignment, with

optional surrounding spaces. If every segment passes this check and at least one pipe was found, it returns 1.

Only if both the current line has pipes and the next line is a valid separator does peek_table() return 1. This two-line check prevents normal text containing pipes from being mistaken for a table.

parse_table() then runs. It calls parse_table_row() for the header row, which advances through the line consuming pipe-separated cells. For each cell, it creates a NODE_TABLE_CELL, calls parse_inline_until() to fill it with inline content up to the next TOK_PIPE, trims trailing whitespace from the cell, and adds it to a NODE_TABLE_ROW. The row is added to the table node as the header.

Next, parse_table_alignment() processes the separator line. It walks through the segment between each pair of pipes and calls alignment_for_segment(), which looks at whether colons appear before the dashes, after, both, or neither, which is mapped to left, right, center, or default left alignment, respectively. The resulting alignment characters are concatenated into a string and stored in the table node's value field.

Then the body rows are consumed one by one with the same parse_table_row() call, looping as long as peek_table_row() returns true. Each row becomes a child of the table node following the header row.

4.5.5 peek_list_marker()

peek_list_marker() is the most complex of the peek functions. Starting from the current parser position, it scans forward accumulating whitespace to measure indentation, then checks if the next non-whitespace token is either a TOK_DASH or a TOK_TEXT token whose value is a number followed by a period. It records the indentation level, whether the list is ordered, and the exact position of the marker token and passes it back through pointer arguments. If no marker is found, it returns 0.

parse_list() is just a thin wrapper that calls parse_list_at() with a base indentation of 0.

parse_list_at() takes a base_indent parameter, which is how it handles nesting. It first calls peek_list_marker() to confirm there's actually a list marker at the expected indentation level. If not, it returns NULL immediately.

Then it enters a loop, continuing as long as peek_list_marker() finds a marker at exactly base_indent. At the start of each iteration, it advances the parser to the marker position by calling advance() in a loop, then advances past the marker itself. A NODE_ITEM is created and parse_inline_until_newline() fills it with the inline content on that line. The newline is consumed.

After the inline content is collected, the item checks whether what follows is nested content. It calls peek_list_marker() again. If the next marker has an indentation greater than base_indent, it

recursively calls parse_list_at() with that deeper indentation level. The resulting nested list node is added as a child of the current item. This recurses as deeply as the indentation goes.

The loop also checks for code blocks that might be nested inside a list item. If a triple backtick is found at a greater indentation, parse_code_block() is called and the result added to the item as well. If neither a nested list nor a code block is found at a greater indent, the inner loop breaks and the item is finalized.

If parse_list_at() is called but the very first peek_list_marker() fails, it returns NULL. This can be either because there's no marker at all or the indentation doesn't match. The caller, which is either parse_list() or the nested content loop, treats NULL as a signal that list parsing is done at that level.

4.6 parser_inline.c

4.6.1 parse_inline()

parse_inline() is the missing link between the block parser and the inline parser. Every block parser that contains readable content calls **parse_inline()** in a loop within its boundary to fill in its children. This includes headings, paragraphs, list items, blockquotes, and table cells. This function is what makes the two step process work. The block parser establishes the container and the stopping condition, and parse_inline() fills it with structured inline nodes. Without this call, every block node would just be an empty shell with no children, and the emitter would have nothing to recurse into.

The notable exceptions are code blocks and math blocks. Neither calls **parse_inline()** at all. Instead they use **make_raw_text_from_tokens()**, which deliberately bypasses all interpretation and stores content as raw flat text. This is intentional because a code block is expected to contain code, not Markdown, so a ** inside it should come out as two asterisks, not bold text. The same applies to math blocks, where the content is LaTeX math syntax that must pass through unmodified. So the presence or absence of parse_inline() is actually what defines the difference between a content block and a raw block. Blocks that call it get a fully interpreted inline subtree. Blocks that do not get flat text and nothing else.

parse_inline() is primarily a dispatcher. It examines the current token, delegates to the appropriate inline parser when possible, and falls back to creating a plain text node for unhandled tokens. It looks at exactly one token, decides which inline parser is responsible for it, and delegates. If it sees TOK_DOUBLE_STAR it calls parse_emphasis(). If it sees TOK_LBRACKET it calls parse_link(). If it sees TOK_BACKTICK it calls parse_inline_code(). If it sees plain TOK_TEXT it calls parse_text(). Each of those sub-parsers then does their own work by consuming or peeking at however many tokens they need, building their own subtree, and returning a completed node. parse_inline() takes that node and immediately attaches it to the parent block node via add_child(), then returns.

Its end goal is to return an ASTNode such that the returned ASTNode can be connected to the parent.

If it finds a TOK_TEXT, that singular word is the whole ASTNode that is returned. If it sees a TOK_DOUBLE_STAR, it translates the need for emphasis, passing the entire tokenlist to the emphasis function to run through and returns the tree that the emphasis function returns later on.

4.6.2 parse_emphasis()

parse_emphasis() is called when parse_inline() encounters any emphasis token, such as TOK_STAR, TOK_DOUBLE_STAR, TOK_TRIPLE_STAR, or their underscore equivalents. **node_type_for_emphasis()** is called first to decide what node type to create. It is a simple switch case that returns NODE_ITALIC for single, NODE_BOLD for double, NODE_BOLD_ITALIC for triple. parse_emphasis() then saves start_pos, consumes the opening delimiter, creates the node, and enters its own inner loop.

The loop continues until it encounters the matching closing delimiter, TOK_NEWLINE, or TOK_EOF. Inside the loop, nested emphasis tokens are handled recursively, plain TOK_TEXT is parsed with parse_text(), and any other token falls back to plain text. Second it checks if the current token is a different emphasis token, such as TOK_STAR inside a TOK_DOUBLE_STAR context. If so, parse_emphasis() calls itself recursively with that new token type, going one level deeper to build a nested emphasis subtree. Thirdly, it checks for plain TOK_TEXT, calling parse_text() to create a NODE_TEXT child. If none of these match, the token is unknown, it is consumed anyway and added as a plain text node via the fallback at the bottom of the loop.

It does not need to store or access the other emphasis tokens, as it only needs to check if its a valid closer for the last opener, because the only valid syntax for a nested emphasis set is when the nested tokens do not overlap.

Once the loop exits it calls match() looking for the closing delimiter. If found, valid is set to 1 and the completed node is returned.

If the loop exited because it hit TOK_NEWLINE or TOK_EOF instead of the closing delimiter, match() fails. At this point parse_emphasis() knows the block boundary was reached with no closing token ever found. It then does three things in order. Firstly, it calls free_ast() on the node it was building, freeing every child that had been collected under it. Secondly, it sets valid to 0 so the caller knows the parse failed. Thirdly, it calls make_raw_text_from_tokens() from start_pos to the current position, which takes every token that was consumed and flattens them all into a single raw NODE_TEXT node. That text node is returned instead of the emphasis node. The ** that triggered the whole attempt comes out as literal asterisks in the output, and all the content that was inside comes out as plain unformatted text. Nothing is lost, the partial node is fully cleaned up, and the caller receives something it can still attach to the parent and move on from.

4.6.3 parse_link()

parse_link() is called when **parse_inline()** sees TOK_LBRACKET. The first thing it does is save start_pos. This is critical because a [alone is not enough to confirm a link. The full [text](url) structure needs to be verified before committing.

It then consumes the [and creates a NODE_LINK. From here it enters its first collection phase, calling **parse_inline_until()** which loops calling **parse_inline()** until it sees TOK_RBRACKET. This is the one exception to the bottom of stack rule, where the rest of the inline parsers are not expected to call for other inline parsers.

The display text inside the brackets goes through full inline parsing because Markdown allows [**bold text**](url) as a valid link. The bold node becomes a child of the link node rather than plain text. Once TOK_RBRACKET is reached the loop stops.

After the display text is collected, the parser checks for TOK_RBRACKET followed immediately by TOK_LPAREN using match(). If either is missing, the link structure is broken. free_ast() is called on the partial node, p->pos is restored to start_pos, and NULL is returned. The [is left unconsumed and parse_inline() falls through to the catch-all, emitting it as plain text.

If both are found, the parser enters its second collection phase for the URL. It advances token by token until TOK_RPAREN, treating everything in that range as raw content.

Unlike the display text, the URL is not passed through parse_inline(). It is collected with make_raw_text_from_tokens() and stored directly into node->value. A URL should never be interpreted as Markdown. If TOK_RPAREN is never found and TOK_EOF is hit instead, the same restore and NULL return applies. On success, the completed NODE_LINK is returned with its display text children and its URL sitting in node->value.

4.6.4 parse_image()

parse_image() is structured identically to **parse_link()** with one difference at the very start. **parse_inline()** only calls this function when it sees TOK_BANG (!) immediately followed by TOK_LBRACKET. It achieves this by peeking one token ahead before deciding.

Once inside, start_pos is saved and both ! and [are consumed. A NODE_IMAGE is created. The alt text inside the brackets is collected with parse_inline_until() the same way as link display text, and becomes children of the image node. Then] and (are checked with match(), and the image path between the parentheses is collected with make_raw_text_from_tokens() and stored in node->value. The same three failure points of missing], missing (, missing) apply and each one calls free_ast(), restores start_pos, and returns NULL.

The node structure ends up the same as a link. Children for the alt text, node->value for the path. The emitter then uses node->value in \includegraphics{} and the alt text children in \textit{} after it.

4.6.5 parse_inline_code()

parse_inline_code() is called when **parse_inline()** sees TOK_BACKTICK. It saves start_pos, consumes the opening backtick, then saves content_start as the position immediately after it. From here it enters a simple advance loop, moving forward one token at a time without looking at or interpreting anything, until it hits TOK_BACKTICK, TOK_NEWLINE, or TOK_EOF.

This function does not call **parse_inline()** or any other inline parser, because inline code content is treated as raw text rather than nested Markdown.

If the closing backtick is found, **make_raw_text_from_tokens()** is called from content_start to p->pos - 1, capturing everything between the two backticks. That raw string is copied into node->value of a fresh NODE_CODE_INLINE and returned. The -1 is to exclude the closing backtick itself from the content range.

If the closing backtick is never found and the loop exits on TOK_NEWLINE or TOK_EOF instead, there is no position restore and no node to free. Instead, **make_raw_text_from_tokens()** is called from start_pos to the current position, returning everything, including the opening backtick as a single flat NODE_TEXT. The backtick comes out as a literal character in the output.

4.6.6 parse_math_inline()

parse_math_inline() is called when **parse_inline()** sees TOK_DOLLAR. It is structured identically to **parse_inline_code()**. start_pos is saved, the opening $ is consumed, content_start is recorded, and a simple advance loop runs until it finds the closing $, TOK_NEWLINE, or TOK_EOF. Like inline code it never calls **parse_inline()** because everything between the dollar signs is treated as raw content.

If the closing $ is found, **make_raw_text_from_tokens()** captures the content from content_start to p->pos - 1 and stores it in node->value of a NODE_MATH_INLINE. The content is stored completely unescaped and unmodified. The idea is that it is already LaTeX math syntax and must pass through to the emitter exactly as written. The emitter then wraps it in $...$ directly.

If the closing $ is never found, the same fallback as inline code applies. **make_raw_text_from_tokens()** from start_pos returns everything, including the opening $ as a flat NODE_TEXT.

4.6.7 parse_text()

parse_text() is the simplest function in the entire parser. It advances past the current TOK_TEXT token, creates a NODE_TEXT, copies the token's value directly into node->value, and returns. One token in, one node out, no children, no loops, no failure cases, no fallback needed. It cannot fail because by the time **parse_inline()** calls it, the token type has already been confirmed as TOK_TEXT.

This is the true bottom of the stack. Every other inline parser eventually produces either a structured node with NODE_TEXT leaves beneath it, or falls back to a raw NODE_TEXT on failure. Either way everything bottoms out here. A NODE_BOLD has NODE_TEXT children. A NODE_LINK has NODE_TEXT children for its display text. A NODE_ITALIC inside a NODE_BOLD has NODE_TEXT children. No matter how deep the nesting goes, the leaves are always plain text nodes created by parse_text() or make_raw_text_from_tokens(). The entire inline subtree under any block node is ultimately a tree of formatting wrappers around plain text at the bottom.

4.7 LaTeX generation

Every parser function, whether block-level or inline-level, returns a completed ASTNode upward to its caller. Inline parsers like parse_text() and parse_emphasis() return nodes to parse_inline(), which attach them as children of whatever block node is currently being built. Block parsers like parse_heading() and parse_list_at() return their completed nodes to parse_block(), which returns them to parse_document(). This bubbling continues until parse_document() has attached every top-level block as a child of the root NODE_DOCUMENT. When the parser's main loop finally hits TOK_EOF, the tree is complete and self-contained.

Once parse_document() returns the root node, it is handed directly to generate_latex(), which is the entry point for the emission stage. This function calls emit_block() on the root and nothing else. From that single call, the entire tree is walked recursively and the LaTeX output is written token by token to the output file. There is no intermediate representation between the AST and the final output, and no parsing logic in the emitter at all. Every structural decision was made during parsing. The emitter only serialises what is already there.

4.8 parser_latex.c

4.8.1 escape_latex()

escape_latex() is a character-by-character scan over a plain text string that substitutes any character with special meaning in LaTeX for its safe equivalent. Backslash becomes \textbackslash{}, curly braces become { and }, and the characters #, $, %, &, _, ^, and ~ each become their backslash-escaped forms. Every other character is written directly. This function is called exclusively from emit_inline() on NODE_TEXT and NODE_CODE_INLINE content. It is never called on math content, code block lines, or URLs, all of which are written raw. This makes escape_latex() the single chokepoint through which all user-facing prose passes before reaching the output file.

4.8.2 emit_inline()

emit_inline() is responsible for all formatting nodes and leaf content. It is called by emit_block() whenever a structural node contains inline children, and it calls itself recursively for nested inline structures.

For NODE_TEXT it calls escape_latex() on the node's value, which is the only point in the entire pipeline where special LaTeX characters are substituted for their escaped equivalents.

For NODE_BOLD it wraps children in \textbf{}, for NODE_ITALIC in \emph{}, and for NODE_BOLD_ITALIC in \textbf{\emph{}}.

For NODE_CODE_INLINE it wraps node->value in \texttt{} and passes it through escape_latex().

For NODE_LINK it writes \href{} with the URL from node->value and the display text children inside the second argument.

For NODE_IMAGE it writes \includegraphics[height=1.5em]{} using node->value as the path, and if the node has children it appends them in \textit{} as the alt text.

For NODE_MATH_INLINE it wraps node->value directly.

For NODE_MATH_BLOCK appearing in an inline context it writes \[and \] around the raw child text lines.

In all math cases the content bypasses escape_latex() entirely, since the content is already valid LaTeX math syntax and must pass through unmodified.

4.8.3 emit_block()

emit_block() is responsible for all structural nodes. It switches on the node's type and handles each case by writing the appropriate LaTeX wrapper and then recursing into children. For NODE_DOCUMENT it writes the full document preamble, including the documentclass declaration, package imports, and paragraph spacing settings, then loops over all children before closing with \end{document}.

For NODE_HEADING it maps the node's level field to the appropriate LaTeX sectioning command, from \section at level one down to \subparagraph at level five, then calls emit_inline() on each child to fill in the heading text.

For NODE_PARAGRAPH it calls emit_inline() on each child and appends a double newline to produce paragraph spacing in the output.

For NODE_BLOCKQUOTE it wraps children in a quote environment, calling emit_block() recursively on each child paragraph inside it.

For NODE_LIST it opens either an itemize or enumerate environment depending on the node's ordered field, then calls emit_block() on each NODE_ITEM child. NODE_ITEM itself falls through to the default case, which simply calls emit_block() on all of its children, meaning both inline content and any nested lists or code blocks are handled by the same recursive call.

For NODE_CODE_BLOCK it opens a verbatim environment and writes each NODE_TEXT child line by line with a trailing newline, bypassing all escaping.

For NODE_MATH_BLOCK it writes the content between [and] with the same raw passthrough.

For NODE_TABLE it reads the alignment string stored in node->value to construct the tabular column specification, writes the header row with \hline, then writes each subsequent row with cells separated by & and rows terminated by \, adding a final \hline after the last row.

5. Examples

This section contains visuals of our implementation of our Markdown-to-Latex compiler. This includes a demo which demonstrates Markdown input being converted to Latex syntax, and the Abstract Syntax Tree (AST) of our parser.

Sample input (Markdown file)

Fig. 1.1: Rendered view of Demo Markdown File

```
# Demo

This is **bold** and *italic* with `code`, [link](http://example.com),
and $x^2 + y^2$.

> Quoted **text**
```

```c
int x = 1;
printf("%d", x);
```

Name	Value
Alpha	42

Fig. 1.2: Demo Markdown file used

Tokens List (output/tokens.txt)

INDEX	TYPE	LINE	COL	VALUE
0	HASH	1	1	[#]
1	TEXT	1	7	[Demo]
2	NEWLINE	1	7	[\n]
3	NEWLINE	2	1	[\n]
4	TEXT	3	5	[This]
5	TEXT	3	8	[is]
6	TEXT	3	9	[ ]
7	DOUBLE_STAR	3	9	[**]
8	TEXT	3	15	[bold]
9	DOUBLE_STAR	3	15	[**]
10	TEXT	3	21	[and]
11	TEXT	3	22	[ ]
12	STAR	3	22	[*]
13	TEXT	3	29	[italic]
14	STAR	3	29	[*]
15	TEXT	3	35	[with]
16	TEXT	3	36	[ ]
17	BACKTICK	3	36	[`]
18	TEXT	3	41	[code]
19	BACKTICK	3	41	[`]
20	TEXT	3	43	[,]
21	TEXT	3	44	[ ]
22	LBRACKET	3	44	[[]
23	TEXT	3	49	[link]
24	RBRACKET	3	49	[]]
25	LPAREN	3	50	[(]
26	TEXT	3	69	[http://example.com]
27	RPAREN	3	69	[)]
28	TEXT	3	71	[,]
29	TEXT	3	75	[and]
30	TEXT	3	76	[ ]
31	DOLLAR	3	76	[$]
32	TEXT	3	80	[x^2]

```
33      TEXT          3       82      [ +]
34      TEXT          3       86      [ y^2]
35      DOLLAR        3       86      [$]
36      TEXT          3       88      [.]
37      NEWLINE       3       88      [\n]
38      NEWLINE       4       1       [\n]
39      GT            5       1       [>]
40      TEXT          5       9       [ Quoted]
41      TEXT          5       10      [ ]
42      DOUBLE_STAR   5       10      [**]
43      TEXT          5       16      [text]
44      DOUBLE_STAR   5       16      [**]
45      NEWLINE       5       18      [\n]
46      NEWLINE       6       1       [\n]
47      TRIPLE_BACKTICK  7       1       [```]
48      TEXT          7       5       [c]
49      NEWLINE       7       5       [\n]
50      TEXT          8       4       [int]
51      TEXT          8       6       [ x]
52      TEXT          8       8       [ =]
53      TEXT          8       11      [ 1;]
54      NEWLINE       8       11      [\n]
55      TEXT          9       7       [printf]
56      LPAREN        9       7       [(]
57      TEXT          9       13      ["%d",]
58      TEXT          9       15      [ x]
59      RPAREN        9       15      [)]
60      TEXT          9       17      [;]
61      NEWLINE       9       17      [\n]
62      TRIPLE_BACKTICK  10      1       [```]
63      NEWLINE       10      4       [\n]
64      NEWLINE       11      1       [\n]
65      PIPE          12      1       [|]
66      TEXT          12      7       [ Name]
67      TEXT          12      8       [ ]
68      PIPE          12      8       [|]
69      TEXT          12      15      [ Value]
70      TEXT          12      16      [ ]
71      PIPE          12      16      [|]
72      NEWLINE       12      17      [\n]
73      PIPE          13      1       [|]
74      DASH          13      2       [-]
75      DASH          13      3       [-]
76      DASH          13      4       [-]
77      DASH          13      5       [-]
78      DASH          13      6       [-]
79      DASH          13      7       [-]
80      PIPE          13      8       [|]
81      DASH          13      9       [-]
82      DASH          13      10      [-]
83      DASH          13      11      [-]
```

```
84      DASH         13      12      [-]
85      DASH         13      13      [-]
86      DASH         13      14      [-]
87      TEXT         13      16      [:]
88      PIPE         13      16      [|]
89      NEWLINE      13      17      [\n]
90      PIPE         14      1       [|]
91      TEXT         14      8       [ Alpha]
92      TEXT         14      9       [ ]
93      PIPE         14      9       [|]
94      TEXT         14      13      [ 42]
95      TEXT         14      14      [ ]
96      PIPE         14      14      [|]
97      NEWLINE      14      15      [\n]
98      EOF          15      1       [EOF]
```

Fig 2: List of tokens extracted by lexer from demo Markdown file

AST generated from the parser

```
DOCUMENT
├── HEADING(level=1)
│   └── TEXT(" Demo")
├── PARAGRAPH
│   ├── TEXT("This")
│   ├── TEXT(" is")
│   ├── TEXT(" ")
│   ├── BOLD
│   │   └── TEXT("bold")
│   ├── TEXT(" and")
│   ├── TEXT(" ")
│   ├── ITALIC
│   │   └── TEXT("italic")
│   ├── TEXT(" with")
│   ├── TEXT(" ")
│   ├── CODE_INLINE("code")
│   ├── TEXT(",")
│   ├── TEXT(" ")
│   ├── LINK("http://example.com")
│   │   └── TEXT("link")
│   ├── TEXT(",")
│   ├── TEXT(" and")
│   ├── TEXT(" ")
│   ├── MATH_INLINE("x^2 + y^2")
│   └── TEXT(".")
├── BLOCKQUOTE
│   └── PARAGRAPH
│       ├── TEXT(" Quoted")
│       ├── TEXT(" ")
│       └── BOLD
│           └── TEXT("text")
├── CODE_BLOCK
```

```
│   ├── TEXT("int x = 1;")
│   └── TEXT("printf(\"%d\", x);")
└── TABLE
    ├── TABLE_ROW
    │   ├── TABLE_CELL
    │   │   ├── TEXT(" Name")
    │   │   └── TEXT(" ")
    │   └── TABLE_CELL
    │       ├── TEXT(" Value")
    │       └── TEXT(" ")
    └── TABLE_ROW
        ├── TABLE_CELL
        │   ├── TEXT(" Alpha")
        │   └── TEXT(" ")
        └── TABLE_CELL
            ├── TEXT(" 42")
            └── TEXT(" ")
```

Fig 3: AST generated from parser

Output LaTeX (output/output.tex)

```
\documentclass{article}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage{lmodern}
\usepackage{hyperref}
\usepackage{graphicx}
\setlength{\parindent}{0pt}
\setlength{\parskip}{0.8\baselineskip}

\begin{document}

\section{ Demo}

This is \textbf{bold} and \emph{italic} with \texttt{code},
\href{http://example.com}{link}, and $x^2 + y^2$.

\begin{quote}
 Quoted \textbf{text}

\end{quote}

\begin{verbatim}
int x = 1;
printf("%d", x);
\end{verbatim}

\begin{tabular}{lr}
\hline
 Name  &  Value  \\
\hline
```

```
 Alpha  &  42  \\
\hline
\end{tabular}

\end{document}
```

Fig 4.1: Generated Latex file output by the LaTeX generation

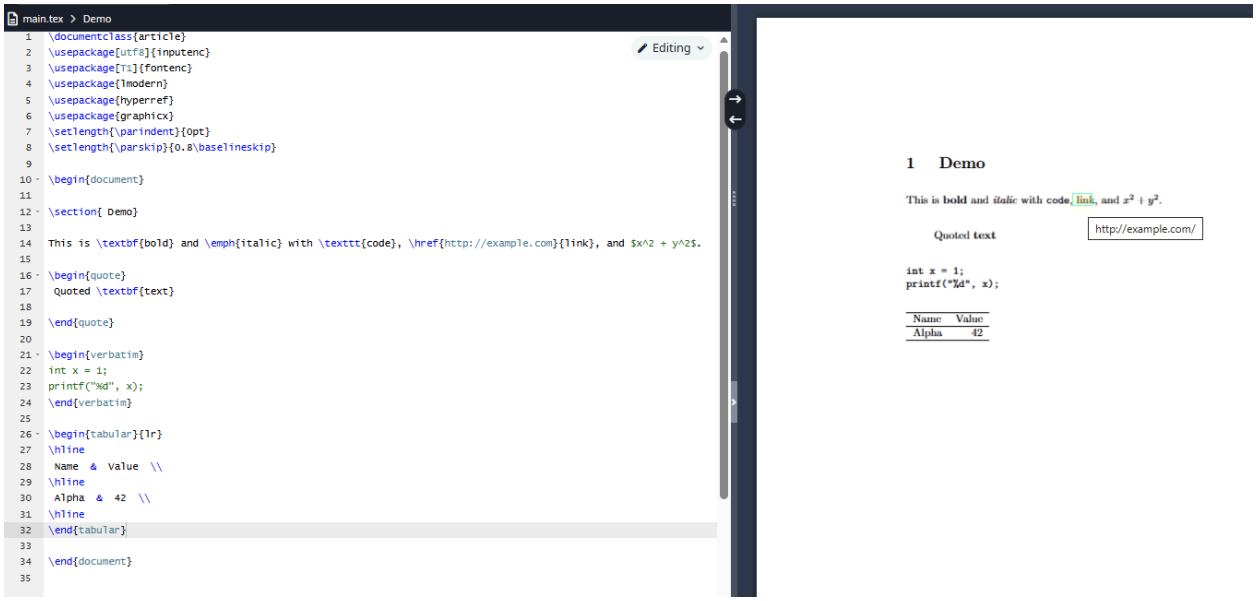

Fig 4.2: Render of generated Latex file

Sample Input II (PoC for item emission bug)

```
# List Item Emission Demo


- plain list item text should be emitted correctly
- **bold text** inside a list item should be emitted inline
- item with a nested list:
  - nested plain text
  - nested `inline code`
- item with a nested code block:
  ```
  int main(void) {
      return 0;
  }
  ```


Normal paragraph after the list.
```

Fig 5: Demo Markdown file used for debugging

Sample Output II (PoC for item emission bug)

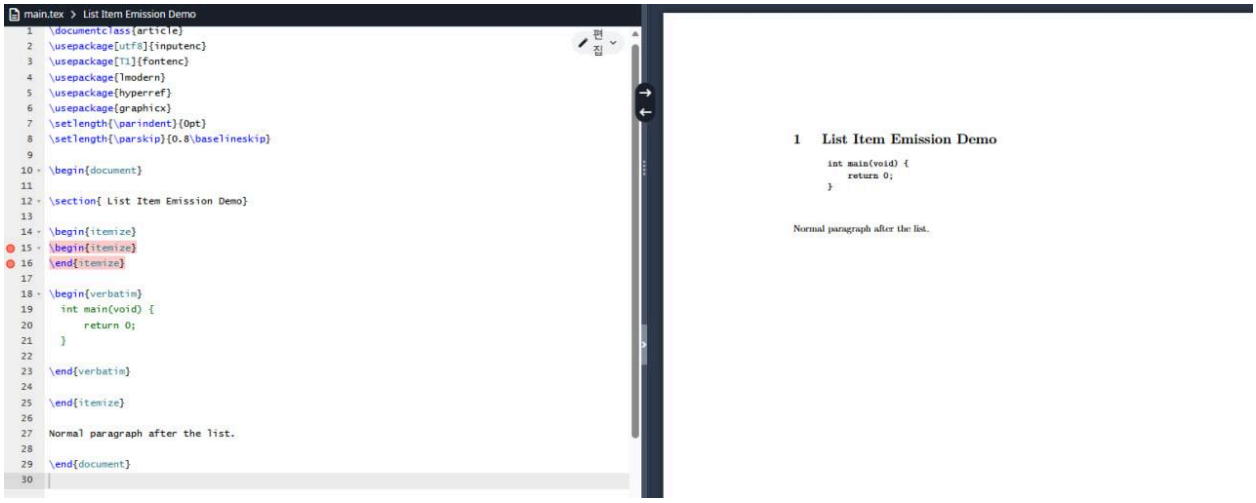

Fig 6.1: Render of generated Latex file (before bugfix)

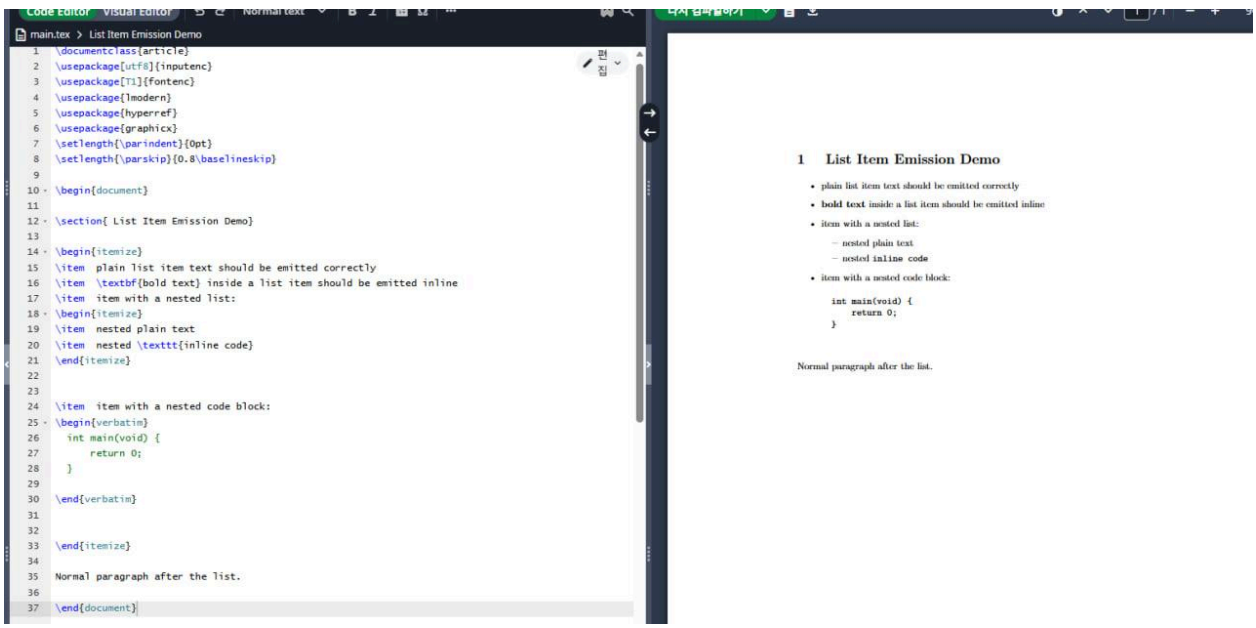

Fig 6.2: Render of generated Latex file (after bugfix)

6. Conclusion

During validation with more complex sample inputs, we observed a limitation in how the current lexer handles backslash characters inside raw content such as code blocks and block math. The lexer currently applies escape-style handling globally, which is suitable for normal Markdown text, but can interfere with raw LaTeX or code content that should be preserved exactly as written. As a result, commands or escape sequences containing backslashes may be altered before they reach the output stage. This does not affect simpler Markdown features such as headings, paragraphs, lists, tables, links, and basic inline formatting, but it becomes visible in more complex cases involving raw code or mathematical expressions.

Joshua contributed a large share of the implementation work across the project and was responsible for much of the core development. Yi Shen helped establish the foundation of the project and contributed to the earlier stages of development, including setup, tokenisation, and supporting code changes. Ming Hui contributed substantially to the parser implementation and other coding work. Jin focused mainly on debugging, testing, report writing and validating fixes during development. Anika contributed to the modularisation of the parser component, testing and validation of the compiler, bug fixes and report writing. Our GitHub repository can be viewed through this link: https://github.com/CoderJae777/Markdown-To-LaTex-Convertor-in-C.git

Throughout this project, we gained practical experience in building a full Markdown-to-LaTeX conversion pipeline in C, covering file handling, lexing, parsing, AST construction, and code generation. We learned the importance of designing clear data structures and separating the lexer, parser, and emitter into distinct stages so that each part could be tested and debugged more easily. The project also strengthened our understanding of recursive parsing, tree-based representations of structured documents, and the need to test edge cases such as nested formatting, lists, tables, and raw blocks. Beyond the technical work, we learned the importance of teamwork, task division, and combining implementation, debugging, testing, and documentation in order to complete a larger software project effectively.